

Entregable del proyecto Adoptme Backend

Documento tecnico para Google Docs

Proyecto: Adoptme Backend API

Imagen Docker: marcoven/adoptme-backend:2.0

1. Estructura del proyecto

El repositorio implementa una API REST organizada por capas. Las rutas reciben las solicitudes HTTP, los controladores coordinan la respuesta, los servicios concentran la logica de negocio y los repositorios/DAO encapsulan el acceso a MongoDB mediante modelos Mongoose.

Arbol de directorios

Comando equivalente: `find . -maxdepth 4`, excluyendo `.git`, `node_modules`, `evidence` y archivos generados.

```
.
├── .github/
│   └── workflows/
│       └── tests.yml
├── public/
│   ├── documents/
│   │   └── .gitkeep
│   ├── img/
│   │   └── .gitkeep
│   └── pets/
│       └── .gitkeep
├── src/
│   ├── config/
│   │   └── logger.js
│   ├── controllers/
│   │   ├── adoptions.controller.js
│   │   ├── mocking.controllers.js
│   │   ├── pets.controller.js
│   │   ├── sessions.controller.js
│   │   └── users.controller.js
│   ├── dao/
│   │   ├── models/
│   │   │   ├── Adoption.js
│   │   │   ├── Pet.js
│   │   │   └── User.js
│   │   ├── Adoption.js
│   │   ├── Pets.dao.js
│   │   └── Users.dao.js
│   ├── docs/
│   │   ├── components/
│   │   │   └── schemas.yaml
│   │   ├── Adoptions.yaml
│   │   ├── curl-examples.md
│   │   ├── Pets.yaml
│   │   ├── Sessions.yaml
│   │   └── Users.yaml
│   ├── dto/
│   │   ├── Pet.dto.js
│   │   └── User.dto.js
│   └── errors/
```

```

├── CustomError.js
├── errorsDictionary.js
├── middlewares/
│   ├── errorHandler.js
│   └── logger.middleware.js
├── mocks/
│   ├── pets.mock.js
│   └── user.mock.js
├── public/
│   └── img/
│       └── 1671549990926-coderDog.jpg
├── repository/
│   ├── AdoptionRepository.js
│   ├── GenericRepository.js
│   ├── PetRepository.js
│   └── UserRepository.js
├── routes/
│   ├── adoption.router.js
│   ├── logger.router.js
│   ├── mocks.router.js
│   ├── pets.router.js
│   ├── sessions.router.js
│   └── users.router.js
├── services/
│   ├── index.js
│   ├── pet.service.js
│   └── user.services.js
├── utils/
│   ├── index.js
│   ├── multer.js
│   └── uploader.js
├── app.js
├── test/
│   ├── files/
│   │   └── test.pdf
│   ├── adoptions.test.js
│   ├── pets.test.js
│   ├── sessions.test.js
│   ├── user.test.js
│   └── users.test.js
├── .dockerignore
├── .env
├── .gitignore
├── .nvmrc
├── Dockerfile
├── Entregable_Adoptme_Backend.docx
├── Entregable_Adoptme_Backend_GoogleDocs.docx
├── errors.log
├── package-lock.json
├── package.json
└── README.md

```

Proposito de archivos y carpetas principales

Elemento	Proposito
src/app.js	Configura Express, Swagger, middlewares, conexion a MongoDB y routers.
src/routes	Define los endpoints HTTP por dominio: users, pets, sessions, mocks, logger y adoptions.
src/controllers	Contiene la capa request/response y delega la logica en servicios.
src/services	Agrupar operaciones de negocio reutilizables.
src/repository	Abstrae operaciones comunes de

	persistencia.
src/dao y src/dao/models	Implementa acceso a datos y esquemas Mongoose.
src/docs	Contiene especificaciones Swagger y ejemplos de cURL.
src/middlewares	Incluye logger de requests y manejador centralizado de errores.
test	Contiene tests funcionales con Mocha, Chai y Supertest.
Dockerfile	Define la imagen productiva multi-stage para ejecutar la API.
README.md	Documentacion reproducible del proyecto.

2. Tests funcionales

Los tests funcionales de adopciones cubren todos los endpoints definidos en `src/routes/adoption.router.js`: listar adopciones, consultar una adopcion por ID y crear una adopcion entre usuario y mascota.

Codigo completo de test/adoptions.test.js

```
import chai from 'chai';
import supertest from 'supertest';
import app from '../src/app.js';
import { faker } from '@faker-js/faker';

const expect = chai.expect;
const requester = supertest(app);

describe('Adoptions Router', () => {
  let userId;
  let petId;
  let adoptionId;

  before(async () => {
    const userMock = {
      first_name: faker.person.firstName(),
      last_name: faker.person.lastName(),
      email: faker.internet.email(),
      age: faker.number.int({ min: 18, max: 70 }),
      password: '123456'
    };

    const userResponse = await requester
      .post('/api/sessions/register')
      .send(userMock);

    expect(userResponse.status).to.equal(200);
    userId = userResponse.body.payload._id;

    const petMock = {
      name: faker.animal.dog(),
      specie: 'Dog',
      birthDate: '2022-01-01'
    };
  });
});
```

```

    });

    const petResponse = await requester
      .post('/api/pets')
      .send(petMock);

    expect(petResponse.status).toEqual(200);
    petId = petResponse.body.payload._id;
  });

  it('Debe crear una adopción correctamente', async () => {
    const response = await requester.post(`/api/adoptions/${userId}/${petId}`);

    expect(response.status).toEqual(200);
    expect(response.body.status).toEqual('success');
    expect(response.body.message).toEqual('Pet adopted');
  });

  it('Debe obtener todas las adopciones y encontrar la adopción creada', async () => {
    const response = await requester.get('/api/adoptions');

    expect(response.status).toEqual(200);
    expect(response.body.status).toEqual('success');
    expect(response.body.payload).toBe.an('array');

    const createdAdoption = response.body.payload.find(adoption => {
      const owner = adoption.owner?._id || adoption.owner;
      const pet = adoption.pet?._id || adoption.pet;
      return owner?.toString() === userId && pet?.toString() === petId;
    });

    expect(createdAdoption).to.exist;
    adoptionId = createdAdoption._id;
  });

  it('Debe obtener una adopción por ID', async () => {
    const response = await requester.get(`/api/adoptions/${adoptionId}`);

    expect(response.status).toEqual(200);
    expect(response.body.status).toEqual('success');
    expect(response.body.payload).to.have.property('_id', adoptionId);
  });

  it('Debe rechazar una adopción si la mascota ya fue adoptada', async () => {
    const response = await requester.post(`/api/adoptions/${userId}/${petId}`);

    expect(response.status).toEqual(400);
    expect(response.body.status).toEqual('error');
    expect(response.body.error).toEqual('Pet is already adopted');
  });

  it('Debe retornar 404 si el usuario no existe', async () => {
    const fakeUserId = '64f000000000000000000001';
    const response = await requester.post(`/api/adoptions/${fakeUserId}/${petId}`);

    expect(response.status).toEqual(404);
    expect(response.body.status).toEqual('error');
  });
});

```

Explicacion de los grupos de tests

- Preparacion: crea un usuario mediante POST /api/sessions/register y una mascota mediante POST /api/pets para obtener IDs validos.
- Creacion de adopcion: valida que POST /api/adoptions/:uid/:pid responda 200, status success y mensaje Pet adopted.
- Listado de adopciones: valida GET /api/adoptions, confirma que el payload sea un array y localiza la adopcion creada.
- Consulta por ID: valida GET /api/adoptions/:aid y verifica que el payload corresponda al ID recuperado.

- Reglas de negocio negativas: valida que una mascota ya adoptada responda 400 y que un usuario inexistente responda 404.

Evidencia de ejecucion de tests

```
> plantilladocumentacion@1.0.0 test
> mocha --exit test/**/*.test.js

◇ injected env (2) from .env // tip: % enable debugging { debug: true }

Adoptions Router
  ✓ Debe crear una adopción correctamente
  ✓ Debe obtener todas las adopciones y encontrar la adopción creada
  ✓ Debe obtener una adopción por ID
  ✓ Debe rechazar una adopción si la mascota ya fue adoptada
  ✓ Debe retornar 404 si el usuario no existe

Pets Router
  ✓ Debe obtener todas las mascotas
  ✓ Debe crear una mascota
  ✓ Debe crear una mascota con datos dinámicos
  ✓ Debe actualizar una mascota
  ✓ Debe eliminar una mascota
  ✓ Debe fallar si faltan datos obligatorios

Sessions Router
  POST /api/sessions/register
    ✓ Debe registrar un usuario correctamente (53ms)
  POST /api/sessions/login
    ✓ Debe loguear un usuario correctamente (55ms)

Users Router
  ✓ Debe crear un usuario (52ms)
  ✓ Debe subir documentos al usuario

Users Router
  ✓ Debe crear un usuario correctamente (52ms)
  ✓ Debe obtener todos los usuarios
  ✓ Debe obtener un usuario por ID
  ✓ Debe actualizar un usuario
  ✓ Debe eliminar un usuario
  ✓ Debe retornar error si el usuario no existe
  ✓ Debe registrar un usuario (53ms)
  ✓ Debe subir documentos al usuario

23 passing (429ms)
```

3. Dockerizacion

Dockerfile completo

```
FROM node:20-alpine AS dependencies
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production

COPY --from=dependencies /app/node_modules ./node_modules
COPY package*.json ./
COPY src ./src
```

```
USER node
EXPOSE 8080
CMD ["node", "src/app.js"]
```

Decisiones de optimizacion

- Imagen base node:20-alpine: reduce el tamaño final respecto a imágenes Debian completas y mantiene compatibilidad con Node 20.
- Multi-stage build: separa la instalación de dependencias de la imagen runner final.
- `npm ci --omit=dev`: instala dependencias reproducibles desde `package-lock.json` y excluye librerías de testing/desarrollo.
- `dotenv` está clasificado como dependencia de producción porque `src/app.js` lo importa durante el arranque del servidor.
- Copia selectiva: la etapa final copia solo `node_modules`, `package*.json` y `src`, evitando archivos innecesarios.
- `NODE_ENV=production`: declara el modo productivo del runtime.
- `USER node`: ejecuta el proceso con usuario no root para reducir privilegios dentro del contenedor.
- `.dockerignore`: evita enviar `node_modules`, `.git` y logs al contexto de build.

Log de construcción de imagen Docker

```
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 346B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/node:20-alpine
#2 ...

#3 [auth] library/node:pull token for registry-1.docker.io
#3 DONE 0.0s

#2 [internal] load metadata for docker.io/library/node:20-alpine
#2 DONE 0.7s

#4 [internal] load .dockerignore
#4 transferring context: 82B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 196.51kB 0.0s done
#5 DONE 0.0s

#6 [dependencies 1/4] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
#6 resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 0.0s done
#6 DONE 0.0s

#7 [dependencies 2/4] WORKDIR /app
#7 CACHED

#8 [dependencies 3/4] COPY package*.json ./
#8 DONE 0.0s

#9 [dependencies 4/4] RUN npm ci --omit=dev
#9 0.815 npm warn deprecated tar@6.2.1: Old versions of tar are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
#9 0.887 npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
#9 0.928 npm warn deprecated nvm@0.0.4: This is NOT the correct nvm. Visit https://nvm.sh and use the curl command to install it.
#9 0.935 npm warn deprecated npmlog@5.0.1: This package is no longer supported.
#9 0.971 npm warn deprecated multer@1.4.5-lts.2: Multer 1.x is impacted by a number of vulnerabilities, which have been patched in 2.x. You should upgrade to the latest 2.x version.
```

```

#9 1.144 npm warn deprecated lodash.isequal@4.5.0: This package is deprecated. Use require('node:util').isDeepStrictEqual
instead.
#9 1.164 npm warn deprecated lodash.get@4.4.2: This package is deprecated. Use the optional chaining (?) operator instead.
#9 1.239 npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-
cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and
powerful.
#9 1.323 npm warn deprecated gauge@3.0.2: This package is no longer supported.
#9 1.833 npm warn deprecated are-we-there-yet@2.0.0: This package is no longer supported.
#9 2.514 npm warn deprecated glob@7.1.6: Old versions of glob are not supported, and contain widely publicized security
vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at
exorbitant rates) by contacting i@izs.me
#9 2.528 npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security
vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at
exorbitant rates) by contacting i@izs.me
#9 6.508
#9 6.508 added 324 packages, and audited 325 packages in 6s
#9 6.508
#9 6.508 30 packages are looking for funding
#9 6.508   run `npm fund` for details
#9 6.582
#9 6.582 27 vulnerabilities (22 moderate, 4 high, 1 critical)
#9 6.582
#9 6.582 To address issues that do not require attention, run:
#9 6.582   npm audit fix
#9 6.582
#9 6.582 To address all issues (including breaking changes), run:
#9 6.582   npm audit fix --force
#9 6.582
#9 6.582 Run `npm audit` for details.
#9 6.582 npm notice
#9 6.582 npm notice New major version of npm available! 10.8.2 -> 11.17.0
#9 6.582 npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.17.0
#9 6.582 npm notice To update run: npm install -g npm@11.17.0
#9 6.582 npm notice
#9 DONE 6.7s

#10 [runner 3/5] COPY --from=dependencies /app/node_modules ./node_modules
#10 DONE 0.2s

#11 [runner 4/5] COPY package*.json ./
#11 DONE 0.0s

#12 [runner 5/5] COPY src ./src
#12 DONE 0.0s

#13 exporting to image
#13 exporting layers
#13 exporting layers 1.1s done
#13 exporting manifest sha256:1e7c5dfb4859684daa34a1d0c8569ebc567d1a074289fc13c5abe6bf0f4cb0db done
#13 exporting config sha256:137bc9af76d1fala7a663ee7907aae4f24d094698fb56a402a835c1085cdfb8a done
#13 exporting attestation manifest sha256:0c64171900adfd61c77816e8928c2c319319e10800f36df7b9e95f6a6f1b45f9 done
#13 exporting manifest list sha256:15d960bb4e8a88b221437cda56bcfdc999f4e41e295d1cb41d71e38b740ef4ed done
#13 naming to docker.io/marcoven/adoptme-backend:2.0 done
#13 unpacking to docker.io/marcoven/adoptme-backend:2.0
#13 unpacking to docker.io/marcoven/adoptme-backend:2.0 0.4s done
#13 DONE 1.5s

```

4. Imagen Docker

Nombre y tag de la imagen generada: marcoven/adoptme-backend:2.0

Evidencia de imagen construida

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
marcoven/adoptme-backend	2.0	15d960bb4e8a	33 seconds ago	279MB

Evidencia de ejecucion del contenedor

NAMES	IMAGE	STATUS	PORTS
adoptme-backend-doc-evidence-2	marcoven/adoptme-backend:2.0	Up 10 seconds	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp

```

◇ injected env (0) from .env // tip: ↵ auth for agents [www.vestauth.com]
{"level":"info","message":"Listening on 8080"}
{"level":"info","message":"MongoDB connected"}

```

5. Ejecucion del proyecto

Construir la imagen Docker

1. Ejecutar `docker build -t marcoven/adoptme-backend:2.0` . desde la raiz del proyecto.

Ejecutar el contenedor

2. Verificar que MongoDB este disponible.
3. Ejecutar `docker run --name adoptme-backend -p 8080:8080 -e PORT=8080 -e MONGO_URI=mongodb://host.docker.internal:27017/adoptme -e JWT_SECRET=superSecretKey marcoven/adoptme-backend:2.0`.
4. Abrir `http://localhost:8080/api/docs/` para verificar Swagger UI.

Correr los tests

5. Instalar dependencias con `npm install`.
6. Configurar `.env`.
7. Ejecutar `npm test`.

Evidencia de ejecucion exitosa

```
% Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed

  0      0    0     0    0     0      0  0 --:--:-- --:--:-- --:--:--    0
100 3106 100 3106    0     0  177k  0 --:--:-- --:--:-- --:--:--  178k
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 3106
ETag: W/"c22-H8FH9nKD8DeX/nvIRrte6ZjP2a4"
Date: Sun, 14 Jun 2026 01:47:44 GMT
Connection: keep-alive
Keep-Alive: timeout=5

<!-- HTML for static distribution bundle build -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

  <title>Swagger UI</title>
  <link rel="stylesheet" type="text/css" href="./swagger-ui.css" >
  <link rel="icon" type="image/png" href="./favicon-32x32.png" sizes="32x32" /><link rel="icon" type="image/png"
href="./favicon-16x16.png" sizes="16x16" />
  <style>
    html
    {
      box-sizing: border-box;
      overflow: -moz-scrollbars-vertical;
      overflow-y: scroll;
    }
    *,
    *:before,
    *:after
    {
      box-sizing: inherit;
    }

    body {
      margin:0;
      background: #fafafa;
    }
  </style>
</head>
<body>
</body>
</html>
```



```

</style>
</head>

<body>

<svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" style="position:absolute;width:0;height:0">
  <defs>
    <symbol viewBox="0 0 20 20" id="unlocked">
      <path d="M15.8 8H14V5.6C14 2.703 12.665 1 10 1 7.334 1 6 2.703 6 5.6V6h2v-.801C8 3.754 8.797 3 10 3c1.203 0 2 .754 2
2.199V8H4c-.553 0-1 .646-1 1.199V17c0 .549.428 1.139.951 1.307l1.197.387C5.672 18.861 6.55 19 7.1 19h5.8c.549 0 1.428-.139
1.951-.307l1.196-.387c.524-.167.953-.757.953-1.306V9.199C17 8.646 16.352 8 15.8 8z"/></path>
    </symbol>

    <symbol viewBox="0 0 20 20" id="locked">
      <path d="M15.8 8H14V5.6C14 2.703 12.665 1 10 1 7.334 1 6 2.703 6 5.6V8H4c-.553 0-1 .646-1 1.199V17c0 .549.428 1.139.951
1.307l1.197.387C5.672 18.861 6.55 19 7.1 19h5.8c.549 0 1.428-.139 1.951-.307l1.196-.387c.524-.167.953-.757.953-1.306V9.199C17
8.646 16.352 8 15.8 8z" M12 8H8V5.6C8 2.703 6.665 1 4 1 1.334 1 0 2.703 0 5.6V6h2v-.801C2 3.754 2.797 3 4 3c1.203 0 2 .754 2
2.199V8H2c-.553 0-1 .646-1 1.199V17c0 .549.428 1.139.951 1.307l1.197.387C2.672 18.861 3.55 19 4.1 19h5.8c.549 0 1.428-.139
4.951-.307l1.196-.387c.524-.167.953-.757.953-1.306V9.199C12 8.646 11.352 8 10.8 8z"/>
    </symbol>

    <symbol viewBox="0 0 20 20" id="close">
      <path d="M14.348 14.849c-.469.469-1.229.469-1.697 0L10 11.819l-2.651 3.029c-.469.469-1.229.469-1.697 0-.469-.469-.469-
1.229 0-1.697l2.758-3.15-2.759-3.152c-.469-.469-.469-1.228 0-1.697.469-.469 1.228-.469 1.697 0L10 8.183l2.651-3.031c.469-.469
1.228-.469 1.697 0 .469.469.469 1.229 0 1.697l-2.758 3.152 2.758 3.15c.469.469.469 1.229 0 1.698z"/>
    </symbol>

    <symbol viewBox="0 0 20 20" id="large-arrow">
      <path d="M13.25 10L6.109 2.58c-.268-.27-.268-.707 0-.979.268-.27.701-.27.969 0l7.83 7.908c.268.271.268.709 0 .979l-7.83
7.908c-.268.271-.701.27-.969 0-.268-.269-.268-.707 0-.979L13.25 10z"/>
    </symbol>

    <symbol viewBox="0 0 20 20" id="large-arrow-down">
      <path d="M17.418 6.109c.272-.268.709-.268.979 0s.271.701 0 .969l-7.908 7.83c-.27.268-.707.268-.979 0l-7.908-7.83c-.27-
.268-.27-.701 0-.969.271-.268.709-.268.979 0L10 13.25l7.418-7.141z"/>
    </symbol>

    <symbol viewBox="0 0 24 24" id="jump-to">
      <path d="M19 7v4H5.83l3.58-3.59L8 6l-6 6 6 6 1.41-1.41L5.83 13H21V7z"/>
    </symbol>

    <symbol viewBox="0 0 24 24" id="expand">
      <path d="M10 18h4v-2h-4v2zM3 6v2h18V6H3zm3 7h12v-2H6v2z"/>
    </symbol>

  </defs>
</svg>

<div id="swagger-ui"></div>

<script src="./swagger-ui-bundle.js"> </script>
<script src="./swagger-ui-standalone-preset.js"> </script>
<script src="./swagger-ui-init.js"> </script>

<style>
.swagger-ui .topbar .download-url-wrapper { display: none } undefined
</style>
</body>

</html>

```

6. README

Contenido completo del README.md actualizado:

```
# Adoptme Backend API
```

API REST para gestionar usuarios, mascotas y adopciones. El proyecto esta desarrollado con Node.js, Express y MongoDB, e incluye documentacion Swagger, tests funcionales con Mocha/Chai/Supertest y dockerizacion optimizada.

Autor: Marco O. Venegas

```
## Tecnologias
```

```
- Node.js 20
- Express
- MongoDB / Mongoose
- Swagger
```

- Docker
- Mocha / Chai / Supertest
- Winston
- Multer
- JWT

Requisitos

- Node.js 20 o superior
- npm
- MongoDB disponible localmente o mediante una URI remota
- Docker Desktop o Docker Engine, solo si se ejecuta con contenedor

Variables de entorno

Crear un archivo `.env` en la raiz del proyecto:

```
```.env
PORT=8080
MONGO_URI=mongodb://127.0.0.1:27017/adoptme
JWT_SECRET=superSecretKey
```
```

Cuando se ejecute desde Docker en macOS o Windows usando una base MongoDB local del host, usar:

```
```.env
MONGO_URI=mongodb://host.docker.internal:27017/adoptme
```
```

Instalacion local

```
```bash
npm install
```
```

Ejecutar el proyecto localmente

```
```bash
npm start
```
```

Servidor disponible en:

```
```text
http://localhost:8080
```
```

Documentacion Swagger:

```
```text
http://localhost:8080/api/docs
```
```

Ejecutar tests

```
```bash
npm test
```
```

La suite incluye tests funcionales para `adoption.router.js`:

- `POST /api/adoptions/:uid/:pid`: crea una adopcion correctamente.
- `GET /api/adoptions`: lista adopciones y permite verificar que existe la adopcion creada.
- `GET /api/adoptions/:aid`: obtiene una adopcion por ID.
- `POST /api/adoptions/:uid/:pid`: rechaza una mascota ya adoptada.
- `POST /api/adoptions/:uid/:pid`: retorna error 404 cuando el usuario no existe.

Docker

El `Dockerfile` usa multi-stage build:

- `node:20-alpine` como imagen base liviana.
- Una etapa `dependencies` instala dependencias con `npm ci --omit=dev`.
- La etapa `runner` copia solo `node_modules`, `package*.json` y `src`.
- `NODE_ENV=production` queda definido en runtime.
- El contenedor ejecuta la app con el usuario no root `node`.
- `.dockerignore` excluye `node_modules`, logs de npm, `.git` y `.gitignore` del contexto de build.

Construir imagen Docker

```
```bash
docker build -t marcoven/adoptme-backend:2.0 .
```
```

Ejecutar contenedor

```
```bash
docker run --name adoptme-backend \
 -p 8080:8080 \
 -e PORT=8080 \
 -e MONGO_URI=mongodb://host.docker.internal:27017/adoptme \
 -e JWT_SECRET=superSecretKey \
 marcoven/adoptme-backend:2.0
```
```

Si MongoDB corre en otro host, reemplazar `MONGO\_URI` por la URI correspondiente.

Imagen Docker

Nombre y tag local:

```
```text
marcoven/adoptme-backend:2.0
```
```

Imagen publicada en Docker Hub:

```
```text
marcoven/adoptme-backend:2.0
```
```

Referencia compartida:

```
```text
https://hub.docker.com/repository/docker/marcoven/adoptme-backend/tags/2.0/sha256-
cbb8ce8b02ca28213cbbd69b7ba07ebf5adcbb1da2fe1f42202f74face623598
```
```

Endpoints principales

```
```text
GET /api/users
POST /api/users

GET /api/pets
POST /api/pets

GET /api/adoptions
GET /api/adoptions/:aid
POST /api/adoptions/:uid/:pid

POST /api/sessions/login
POST /api/sessions/register
```
```

Estructura general

```
```text
Routes -> Controllers -> Services -> Repository -> DAO -> Models -> Database
```
```

Carpetas principales:

- `src/app.js`: configura Express, middlewares, Swagger, conexion a MongoDB y routers.
- `src/routes`: define endpoints HTTP por modulo.
- `src/controllers`: maneja request/response y coordina servicios.
- `src/services`: contiene logica de negocio.
- `src/repository`: abstrae operaciones de persistencia.
- `src/dao`: acceso a datos y modelos Mongoose.
- `src/docs`: especificaciones Swagger y ejemplos.
- `src/middlewares`: logger y manejo centralizado de errores.
- `test`: tests funcionales con Mocha, Chai y Supertest.
- `public`: destino de archivos subidos.